FIELD DOSSIER // BACKEND-EX-12 // REV. 2026

12 APIs

Every Backend Developer Should Build Once

Not another to-do-list roadmap. Twelve real specifications — stack, strategy, and the exact trap engineers fall into on attempt one — for the systems that actually get asked about in a system design interview.

01 SHORTENER · 02 RATE-LIMIT · 03 LEDGER · 04 WEBHOOK · 05 AUTH/RBAC · 06 FILE-PIPE
07 REALTIME · 08 SCHEDULER · 09 SEARCH · 10 AUDIT-LOG · 11 GATEWAY · 12 CHAT

COMPILED BY

@dipanshu.decoded

STATUS: BUILD-READY
CLASSIFICATION: PUBLIC

BRIEFING // WHY THIS LIST IS DIFFERENT

This is not the list you've already seen a hundred times

01

These aren't CRUD wrappers

A todo-list API teaches routing. None of these twelve let you finish without confronting retries, race conditions, or a decision you can't undo in production.

02

Each one maps to a real hiring bar

Idempotent payments, webhook delivery, and multi-tenant auth are exactly what interviewers probe for at real companies — this list is reverse-engineered from that bar, not from what's trendy.

03

A named stack, not a toy one

Every spec below names the language, database, and infra you'd actually reach for — the tech choice is part of the engineering decision, not an afterthought.

04

Depth over breadth, on purpose

Building four of these properly teaches more than shipping forty half-finished projects that all stop the moment they return 200 OK.

Each spec ahead gives you what it teaches, the exact stack, the build strategy, and the mistake almost everyone makes on attempt one.

MANIFEST // TWELVE SPECIFICATIONS

Twelve builds, each exposing a different production failure mode

01 **URL Shortener + Analytics**
Base62 encoding, collision handling, click-stream at scale.

02 **Rate Limiter as a Service**
Token bucket in Redis, served as infrastructure.

03 **Idempotent Payment Ledger**
Double-entry books, keys that survive retries.

04 **Webhook Delivery System**
Signed payloads, backoff, dead-letter recovery.

05 **Multi-Tenant Auth & RBAC**
Row-level isolation, scoped tokens.

06 **Async File Processing**
Presigned uploads, background workers.

07 **Real-Time Notifications**
WebSockets, SSE fallback, pub-sub fan-out.

08 **Distributed Job Scheduler**
Leader election so jobs never double-fire.

09 **Full-Text Search API**
Tokenized ranking without an oversized stack.

10 **Event-Sourced Audit Log**
Append-only history that rebuilds state.

11 **API Gateway / BFF**
One client call, parallel fan-out behind it.

12 **Reliable Chat & Messaging**
Delivery guarantees, offline queuing, order.

01

ENCODING / SCALE

— STACK —

- Go or Node.js
- PostgreSQL / Redis
- Base62 encoder
- Kafka or Redis Streams

URL Shortener + Analytics

Everyone builds a URL shortener. Almost nobody builds the analytics half — and that half is where the real engineering lives, because a redirect is trivial and one that scales to millions of clicks without slowing down is not.

Short code generation without collisions

Base62-encode an auto-incrementing ID, or hash-and-truncate with a collision-retry loop — reason about the birthday-paradox math before picking one at scale.

Redirect stays hot, analytics doesn't

The redirect must respond in single-digit milliseconds; log clicks to a queue and process analytics asynchronously so a slow write never delays the redirect.

Strategy — separate read and write paths

Cache short-code lookups aggressively; URL shortener traffic is read-heavy by orders of magnitude — writes are rare, reads are relentless.

Trap on attempt one

A synchronous write-through path means every redirect blocks on an analytics insert — the moment traffic spikes, the 'simple' endpoint becomes the slowest one.

WHY BUILD THIS ONCE

The real lesson isn't shortening URLs — it's separating a hot, latency-critical path from a high-volume, latency-tolerant one.

Rate Limiter as a Service

02

INFRASTRUCTURE

— STACK —

- Go or Rust
- Redis + Lua scripts
- gRPC / lightweight HTTP

Most developers bolt rate limiting on as middleware in one app. Building it as its own service — one other APIs call before processing — teaches what shared infrastructure actually means.

Token bucket, done atomically

Implement check-and-decrement as a single Lua script in Redis, so concurrent requests can't race each other into a double-pass.

Design it as a call other services make

Expose one fast 'is this key allowed right now' endpoint any service can call in milliseconds — rate limiting becomes infrastructure, not duplicated logic.

Strategy — fail open by default

If the limiter itself goes down, decide deliberately whether to block or allow traffic — for most products, failing open is the safer default.

Trap on attempt one

Splitting the check and the decrement into two Redis calls — under real concurrency, this silently lets more requests through than the limit allows.

WHY BUILD THIS ONCE

This teaches the gap between 'a feature in my app' and 'a service my whole system depends on' — a different reliability bar entirely.

Idempotent Payment Ledger

03

FINANCIAL SYSTEMS

— STACK —

- Python or Java
- PostgreSQL (SERIALIZABLE)
- Idempotency-key middleware

This is the API that separates people who've read about ACID from people who've had to defend a balance that must never, under any retry or network blip, be wrong.

Double-entry ledger, not one balance column

Every transaction writes a debit and a credit row instead of mutating one field — this makes the ledger auditable and makes bugs visible instead of silently corrupting a number.

Idempotency keys on every mutating call

A client sends a unique key per attempt; the API returns the original result on retry instead of processing the payment twice.

Strategy — serializable transactions where money moves

Use the strictest isolation level specifically for balance-changing operations — the performance cost is worth it for the one part that truly can't be wrong.

Trap on attempt one

Reading a balance, then writing a new one in two separate steps — under concurrency, this is a textbook lost-update bug that lets a balance go negative.

WHY BUILD THIS ONCE

Build this correctly and you can defend your reasoning to any engineer who's actually shipped a payments system.

Webhook Delivery System

04

RELIABILITY

— STACK —

- Node.js or Go
- Redis or SQS queue
- HMAC-SHA256 signing
- Postgres delivery log

Sending an HTTP request when something happens is easy. Guaranteeing it arrives, proving it came from you, and recovering when the receiver is down for an hour is the actual product.

Signed payloads receivers can trust

Sign every payload with HMAC using a per-customer secret — without this, anyone can forge a webhook and impersonate your system.

Exponential backoff with jitter

Retry with increasing delay plus randomness — fixed intervals cause every failed webhook to retry in lockstep and hammer a recovering endpoint at once.

Strategy — a dead-letter queue, not silent failure

After a fixed number of retries, move the event to a dead-letter queue with full history instead of dropping it.

Trap on attempt one

Retrying forever with no cap, letting one permanently-broken endpoint consume retry workers that should be delivering to everyone else.

WHY BUILD THIS ONCE

A webhook system's job isn't sending requests — it's making a promise about delivery and keeping it, including when the other side fails.

Multi-Tenant Auth & RBAC

05

SECURITY / DATA MODEL

— STACK —

- NestJS or Django
- Postgres row-level security
- JWT with tenant + role claims

Any auth tutorial gets you login and a token. Multi-tenancy is where the real question shows up: how do you guarantee, at the data layer, tenant A can never touch tenant B's row.

Isolation enforced at the database

Row-level security policies keyed on tenant_id mean even a forgotten WHERE clause can't leak another tenant's data — defense in depth, not just discipline.

Scoped tokens carrying tenant and role

Encode tenant ID and role in the token so every request can be authorized without an extra round-trip to figure out permissions.

Strategy — roles as a table, not a hardcoded enum

Model roles and permissions as data so a tenant can eventually define custom roles, avoiding a painful migration later.

Trap on attempt one

Filtering by tenant_id only in application queries — one missed filter in one endpoint becomes a cross-tenant data leak.

> **WHY BUILD THIS ONCE**
>
> **The real skill is isolation that survives a mistake in your own application code, not just correct code.**

06

ASYNC SYSTEMS

— STACK —

- FastAPI or Node.js
- S3-compatible + presigned URLs
- Celery, BullMQ, or SQS

Async File Processing

Uploading to your own server and processing inline works in a demo and falls over the moment a file is large. This teaches the pattern almost every real upload feature uses.

Presigned URLs, files never touch your server

Generate a short-lived, direct-to-storage upload URL — your API coordinates the upload instead of proxying gigabytes through your own process.

Processing happens in a worker

The API accepts an upload notification, enqueues a job, and returns immediately — the actual work happens off the request thread.

Strategy — polling or webhooks, never blocking

Give the client a job ID to poll or let them register a callback — never hold an HTTP connection open for a five-minute transcode.

Trap on attempt one

Processing synchronously inside the upload request — fine in local testing, times out constantly once file sizes grow in production.

WHY BUILD THIS ONCE

The lesson is architectural, not file-specific: any slow operation belongs off the request-response cycle.

07

REAL-TIME SYSTEMS

— STACK —

- Node.js or Go
- Redis Pub/Sub fan-out
- Server-Sent Events fallback

Real-Time Notifications

A polling endpoint checking every few seconds is a workaround, not a real-time system. This forces you to hold open connections and push, not pull.

WebSockets primary, SSE as fallback

WebSockets give a persistent two-way connection; SSE is a simpler one-way alternative for clients that can't maintain one — support both deliberately.

Fan-out through pub-sub

Publish events to a channel and let every instance holding a relevant connection pick them up — this is what makes notifications work across multiple servers.

Strategy — connection state lives outside the process

Track which users connect to which instance in shared storage, not in-memory on one node.

Trap on attempt one

Building against a single instance in dev, then discovering a user's socket is on server A while the event published from server B.

WHY BUILD THIS ONCE

Real-time systems are hard because state has to work correctly across multiple machines — that's the actual skill this teaches.

Distributed Job Scheduler

08

DISTRIBUTED SYSTEMS

— STACK —

- Go
- Postgres advisory locks or etcd
- Redis job locking

A single cron job is fine until you run three replicas for redundancy — and all three fire the same job at once. This is about scheduling correctly under redundancy.

Leader election for scheduling

Use a distributed lock so exactly one running instance is responsible for triggering scheduled jobs at any moment.

Job locking, separate from scheduling

Even with one scheduler, guard actual execution with its own lock to protect against a job still running when the next trigger fires.

Strategy — schedule state in the database

Next-run times and retry counts belong in persistent storage so a restart doesn't lose track or re-trigger everything.

Trap on attempt one

A naive cron loop inside every instance of a scaled service — guarantees duplicate execution past one instance.

> WHY BUILD THIS ONCE
>
> **This makes 'exactly once' concrete instead of theoretical — you'll feel exactly why distributed coordination is hard.**

09

DATA RETRIEVAL

— STACK —

- Postgres tsvector + GIN
- or Elasticsearch / Meilisearch at scale

Full-Text Search API

Search feels solved until you build it and realize ranking and typo tolerance are each their own decision — not something LIKE '%query%' gets you close to.

Tokenized indexes, not substring matching

A proper index tokenizes content with a GIN or inverted index — LIKE degrades linearly with table size and can't rank relevance at all.

Ranking is a deliberate scoring decision

Decide explicitly how term frequency, field weight, and recency combine into a score — ranking is what makes results useful, not just matching.

Strategy — start with Postgres, graduate when needed

Postgres full-text search handles real scale; reach for Elasticsearch only when you need faceting or multi-node sharding.

Trap on attempt one

Reaching for a full Elasticsearch cluster on day one for a dataset that would search fine in Postgres.

WHY BUILD THIS ONCE

Search teaches that 'find matches' and 'find the most relevant matches' are entirely different problems — most tutorials only solve the first.

10

DATA MODELING

— STACK —

- Any backend language
- Postgres append-only or Kafka
- Snapshot table for reads

Event-Sourced Audit Log

Most CRUD apps overwrite history the moment a row updates. This flips that: store every event that happened and derive state from it.

Append-only events as source of truth

Every change is a new, immutable event row — nothing updates or deletes — so you can reconstruct the system at any past moment.

Current state as a derived projection

Compute 'current' by replaying events, cached in a snapshot table for fast reads — the snapshot is an optimization, the log is the truth.

Strategy — version your event schema from day one

Include an event type and version in every record since shapes will inevitably change and old events must still replay correctly.

Trap on attempt one

Treating the snapshot as the real data and the log as an afterthought — you've quietly rebuilt a mutable database with extra steps.

WHY BUILD THIS ONCE

This teaches a genuinely different mental model — state as a consequence of history, not a fact you overwrite.

11

ARCHITECTURE

— STACK —

- Node.js or Go
- Redis response cache
- GraphQL or custom aggregation layer

API Gateway / BFF

Real products rarely need one backend call per screen. A backend-for-frontend teaches you to own that aggregation instead of pushing it onto the client.

One client call, several backend calls behind it

The client hits one endpoint shaped for its screen; the gateway fans out internally and assembles the response — clients never learn your service topology.

Parallel calls, not sequential

Fetch independent data concurrently rather than one after another — sequential fan-out turns a 50ms call into a 250ms one for no reason.

Strategy — cache at the aggregation layer

Cache the assembled response, not just individual upstream responses, when the combination is requested often.

Trap on attempt one

Letting one slow, non-critical upstream call block the entire response — set per-call timeouts and return partial data with a flag instead.

WHY BUILD THIS ONCE

A gateway's job is absorbing your backend's complexity so the client never has to — a genuinely different skill from building one service well.

12

MESSAGING SYSTEMS

— STACK —

- Node.js or Elixir
- Postgres message history
- Redis presence + queuing

Reliable Chat & Messaging

A demo where two tabs message each other isn't the hard part. The hard part is a message sent while offline, delivered correctly once reconnected, exactly once, in order.

Message state machine: sent, delivered, read

Track explicit states instead of a single boolean — this powers the delivery and read receipts every real chat product shows.

Offline delivery via a per-user queue

Messages to an offline user queue and flush on reconnect, instead of being lost or requiring the sender to know connection status.

Strategy — ordering per conversation, not global

Guarantee order within a thread using a monotonic sequence number — global ordering is unnecessary complexity.

Trap on attempt one

Relying on client-side timestamps to order messages — clock drift silently produces out-of-order display for no visible reason.

WHY BUILD THIS ONCE

Chat looks like CRUD from the outside and is actually a delivery-guarantee problem underneath.

SIGN-OFF

You don't need all twelve to level up

Each spec was picked because it's impossible to finish without confronting a real production failure mode — a race condition, a delivery guarantee, an isolation boundary. Build even three or four of these properly, with the stack and strategy actually applied, and you'll have a portfolio that answers system design questions before they're asked.

Don't build all twelve as toy demos. Pick one, apply the strategy notes, break it on purpose, then fix it. That's the difference between a GitHub repo and an engineer who's felt these failure modes firsthand.

COMPILED BY

@dipanshu.decoded

Backend Engineering · System Design